

# Kria KR260 PoC 튜토리얼: SD 카드 기반 A/B 이중화 부팅 구현 가이드

## 튜토리얼 개요

본 문서는 선행 보고서 "Zynq UltraScale+ RFSoc DFE를 위한 2계층 A/B 이중화 부팅... 구현 백서"에서 제시한 2계층 이중화 아키텍처의 핵심 원리를 AMD Kria KR260 로보틱스 스타터 키트에 맞게 변형하여 구현하는 실습형(PoC) 튜토리얼입니다.

### KR260 아키텍처의 주요 차이점 및 PoC 전략:

선행 보고서는 전체 시스템(부트로더, 커널, RootFS)이 단일 QSPI 플래시에 상주하는 RFSoc DFE 보드를 가정했습니다. 하지만 Kria KR260 스타터 키트는 다른 부팅 방식을 사용합니다<sup>1</sup>:

1. **1단계 (QSPI):** BOOT.BIN (FSBL, PMUFW, ATF, U-Boot 포함)은 SOM에 내장된 QSPI 플래시에서 로드됩니다. Kria는 이미 이 QSPI에 대한 A/B 이중화 부팅 펌웨어("Image Selector")를 공장 출하 시 탑재하고 있습니다.<sup>1</sup>
2. **2단계 (SD 카드):** QSPI의 U-Boot가 실행된 후, SD 카드<sup>2</sup>의 FAT 파티션에서 boot.scr, Linux 커널(Image), 디바이스 트리(system.dtb)를 로드하여 부팅합니다.<sup>1</sup>

이러한 Kria의 표준 부트 플로우를 활용하여, 본 튜토리얼은 **\*\*1계층(Tier 1)\*\***은 Kria의 안정적인 QSPI 부트로더를 그대로 사용하고, **2계층(Tier 2)** A/B 이중화(커널, RootFS, 디바이스 트리)를 SD 카드 상에 구현하는 데 집중합니다. 이는 PoC 구현이 용이하고 Kria 보드의 보증을 무효화하지 않는 가장 안전한 접근 방식입니다.

본 튜토리얼을 완료하면, 부팅 실패, 커널 패닉, 시스템 행(hang) 발생 시 SD 카드의 'B' 파티션으로 자동 페일오버하고, 리눅스 유저스페이스에서 다음 업데이트를 준비할 수 있는 강력한 이중화 시스템을 KR260 보드에 구축하게 됩니다.

---

## 1단계: PetaLinux 프로젝트 생성 (Kria KR260 BSP 기준)

먼저 Kria KR260 BSP(Board Support Package)를 사용하여 PetaLinux 프로젝트를 생성합니다. (PetaLinux 2022.1 이상 버전 권장)

1. AMD 웹사이트에서 Kria KR260 BSP 파일을 다운로드합니다.  
(xilinx-kr260-starter-kit-v202X.X-final.bsp)
2. PetaLinux 환경을 설정합니다.  
Bash  
`source <petalinux_install_dir>/settings.sh`
3. BSP를 사용하여 PetaLinux 프로젝트를 생성합니다.  
Bash  
`petalinux-create -t project -s /path/to/xilinx-kr260-starter-kit-v202X.X-final.bsp -n kria_ab_boot`  
`cd kria_ab_boot`
4. PetaLinux 프로젝트 설정을 BSP 기본값으로 구성합니다. `petalinux-config`가 자동으로 실행됩니다. Subsystem AUTO Hardware Settings 메뉴가 Kria SOM을 올바르게 참조하는지 확인하고, 별다른 변경 없이 저장하고 종료합니다.

## 2단계: A/B 부팅을 위한 SD 카드 파티션 준비

본 PoC에서는 SD 카드를 A/B 시스템 파티션으로 구성합니다. PetaLinux의 `petalinux-package --wic` 명령<sup>2</sup>을 사용하여 이미지를 생성할 수도 있지만, PoC의 명확성을 위해 이미지를 빌드한 후 호스트 PC에서 수동으로 파티션을 구성합니다.

1. SD 카드 파티션 레이아웃 설계 (16GB 이상 권장):
  - p1 (boot): 500MB, vfat (FAT32). boot.scr, Image-A, Image-B, system-A.dtb, system-B.dtb 저장.
  - p2 (rootfs\_a): 6GB, ext4. 슬롯 A의 루트 파일 시스템.
  - p3 (rootfs\_b): 6GB, ext4. 슬롯 B의 루트 파일 시스템.
  - p4 (data): 나머지 공간, ext4. 사용자 데이터 (업데이트 시 유지).
2. 호스트 PC에서 SD 카드 포맷 (fdisk 사용):
  - SD 카드를 호스트 리눅스 PC에 삽입하고 `lsblk`로 장치 이름(예: `/dev/sdc`)을 확인합니다.
  - `sudo fdisk /dev/sdc` 명령으로 파티션 테이블을 생성합니다. (기존 파티션은 d로 모두 삭제)
  - `n -> p -> 1 -> (Enter) -> +500M -> t -> c (W95 FAT32 (LBA))`
  - `n -> p -> 2 -> (Enter) -> +6G`
  - `n -> p -> 3 -> (Enter) -> +6G`
  - `n -> p -> 4 -> (Enter) -> (Enter) (나머지 전체)`
  - `w`를 눌러 변경 사항을 저장합니다.
3. 파일 시스템 생성:
  - Bash

```
sudo mkfs.vfat -F 32 -n "boot" /dev/sdc1
sudo mkfs.ext4 -L "rootfs_a" /dev/sdc2
sudo mkfs.ext4 -L "rootfs_b" /dev/sdc3
sudo mkfs.ext4 -L "data" /dev/sdc4
```

### 3단계: U-Boot 및 RootFS 설정 (A/B 로직 활성화)

A/B 로직을 위한 핵심 U-Boot 기능과 리눅스 유틸리티를 활성화합니다.

#### 1. U-Boot 설정 (bootcount 활성화):

- petalinux-config -c u-boot 명령으로 U-Boot menuconfig를 엽니다.
- 다음 항목을 활성화(선택)합니다:
  - Boot options ---> [\*] Enable bootcount limit support<sup>3</sup>
- 참고: KR260 U-Boot 환경은 QSPI의 bootenv 파티션에 저장되도록 사전 설정되어 있습니다.
- menuconfig를 저장하고 종료합니다.

#### 2. RootFS 설정 (u-boot-tools 추가):

- petalinux-config -c rootfs 명령으로 RootFS menuconfig를 엽니다.
- 다음 항목을 활성화합니다:
  - Filesystem Packages ---> misc ---> u-boot-tools ---> [\*] u-boot-tools
  - Filesystem Packages ---> misc ---> u-boot-tools ---> [\*] u-boot-tools-fw-utils<sup>5</sup>
- (PetaLinux 2022.1+) libubootenv가 u-boot-tools-fw-utils를 대체할 수 있습니다. user packages --->에서 libubootenv와 libubootenv-bin을 활성화합니다.<sup>6</sup>
- menuconfig를 저장하고 종료합니다.

### 4단계: 핵심 boot.scr A/B 페일오버 스크립트 작성 (Tier 2 로직)

이 스크립트가 2계층(Tier 2) A/B 부팅 로직의 핵심입니다. Kria U-Boot는 SD 카드의 boot.scr를 자동으로 로드합니다.<sup>1</sup>

1. project-spec/meta-user/recipes-bsp/u-boot/ 디렉토리로 이동합니다. (없으면 mkdir -p로 생성)
2. u-boot-zynq-scr 디렉토리를 생성합니다: mkdir u-boot-zynq-scr
3. u-boot-zynq-scr 디렉토리 안에 A/B 로직을 담을 boot.cmd 파일을 생성합니다.  
vi project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr/boot.cmd  
[코드 스니펫](#)

```
#
# Kria KR260 A/B Boot Script (Tier 2 Logic for SD Card)
#
echo "## Kria A/B Boot Script Initializing..."
```

```
# --- 1. 변수 초기화 ---
# boot_order: 부팅 시도 순서 (A 또는 B)
# boot_a_left/boot_b_left: 슬롯별 부팅 시도 횟수
# bootlimit: bootcount(WDT 리셋 카운터) 최대 허용치
test -n "${boot_order}" |
```

```
| setenv boot_order "A B"
test -n "${boot_a_left}" |
| setenv boot_a_left 3
test -n "${boot_b_left}" |
| setenv boot_b_left 3
test -n "${bootlimit}" |
| setenv bootlimit 3
```

```
# --- 2. 부팅 실패 감지 (WDT 리셋) ---
# U-Boot가 CONFIG_BOOTCOUNT_LIMIT에 의해 'bootcount'를 자동 증가시킴 [8, 3]
if test ${bootcount} -gt 0; then
    echo "WARN: Boot failure detected (WDT Reset? Count: ${bootcount})."
```

```
    # 'boot_order_first' (직전 부팅 시도 슬롯)의 시도 횟수를 1 차감
    setenv current_slot ${boot_order_first}
    if test "${current_slot}" = "A"; then
        setexpr boot_a_left ${boot_a_left} - 1
        echo "Slot A tries remaining: ${boot_a_left}"
    elif test "${current_slot}" = "B"; then
        setexpr boot_b_left ${boot_b_left} - 1
        echo "Slot B tries remaining: ${boot_b_left}"
    fi
```

```
    # bootcount가 bootlimit에 도달하면 Tier 1(QSPI) 복구를 시도
    # (본 PoC에서는 간단히 변수 초기화 후 리셋)
    if test ${bootcount} -ge ${bootlimit}; then
```

```

    echo "FATAL: Boot failure limit reached. Resetting to Golden (Slot A)."
```

```

    run altbootcmd
else
    # 다음 시도를 위해 bootcount만 0으로 리셋하고, 차감된 slot_left 값 저장
    setenv bootcount 0
    saveenv
fi
fi
```

```

# altbootcmd: 모든 시도가 실패했을 때의 최종 복구 명령
# Kria QSPI 부트로더는 이미 이중화되어 있으므로 ,
# 여기서는 SD카드의 A슬롯(Golden)으로 부팅하도록 변수만 초기화
setenv altbootcmd "echo Triggering Tier 2 Fallback to Slot A...; \
    setenv boot_order 'A B'; \
    setenv boot_a_left 3; \
    setenv boot_b_left 3; \
    setenv bootcount 0; \
    saveenv; \
    reset"
```

```

# --- 3. 부팅 가능한 슬롯 선택 ---
```

```

setenv bootargs_set 0
for slot in ${boot_order}; do
    if test ${bootargs_set} -eq 0; then
        echo "Attempting to boot slot ${slot}..."
        if test "${slot}" = "A"; then
            if test ${boot_a_left} -gt 0; then
                echo "Found valid slot A. Tries left: ${boot_a_left}"
                setenv kernel_image "Image-A"
                setenv fdt_image "system-A.dtb"
                setenv rootfs_part "/dev/mmcb1k1p2"
                setenv slot_id "A"
                setenv bootargs_set 1
            fi
            elif test "${slot}" = "B"; then
                if test ${boot_b_left} -gt 0; then
                    echo "Found valid slot B. Tries left: ${boot_b_left}"
                    setenv kernel_image "Image-B"
                    setenv fdt_image "system-B.dtb"
                    setenv rootfs_part "/dev/mmcb1k1p3"
                    setenv slot_id "B"
                    setenv bootargs_set 1
                fi
            fi
        fi
    fi
done
```

```

    fi
fi
done

# --- 4. 선택된 슬롯으로 부팅 시도 ---
if test ${bootargs_set} -eq 1; then
    echo "Loading Kernel (${kernel_image}) and DTB (${fdt_image})..."

    # Kria는 SD 카드의 1번 파티션(FAT)에서 로드
    if fatload mmc 1:1 ${kernel_addr_r} ${kernel_image}; then
        if fatload mmc 1:1 ${fdt_addr_r} ${fdt_image}; then
            # 부팅 인자(bootargs) 설정 [9]
            # Kria 기본 bootargs에 root= 및 rauc.slot= 추가
            setenv bootargs "console=ttyPS0,115200 earlycon clk_ignore_unused cpuidle.off=1 \
                root=${rootfs_part} rootwait rw \
                rauc.slot=${slot_id}"

            # 'boot_order_first'에 현재 시도하는 슬롯을 기록 (실패 감지용)
            setenv boot_order_first ${slot_id}
            saveenv

            echo "Booting Kernel..."
            bootm ${kernel_addr_r} - ${fdt_addr_r}

            # bootm 실패 시 (이미지 손상 등)
            echo "ERROR: Kernel bootm command failed! Triggering reset..."
            reset
        else
            echo "ERROR: Failed to load DTB (${fdt_image})!"
        fi
    else
        echo "ERROR: Failed to load Kernel (${kernel_image})!"
    fi

    # 커널/DTB 로드 실패 시, 리셋하여 bootcount 로직 재시도
    echo "Resetting to retry..."
    reset

# --- 5. 부팅 가능한 슬롯 없음 ---
else
    echo "FATAL: No valid boot slots (A or B) remaining!"
    run altbootcmd
fi

```

'''

#### 4. boot.cmd를 PetaLinux 빌드에 통합:

- project-spec/meta-user/recipes-bsp/u-boot/ 디렉토리에 u-boot-zynq-scr.bbappend 파일을 생성합니다.<sup>10</sup>
- vi project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr.bbappend

코드 스니펫

```
# This bbappend file tells PetaLinux to use our custom boot.cmd
# instead of the default one.[11, 12]
```

```
FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:"
SRC_URI_append = " file://boot.cmd"
```

```
# Force override of the default boot.cmd
do_compile_prepend() {
    cp ${WORKDIR}/boot.cmd ${S}/boot.cmd.default.initrd
}
```

### 5단계: Linux 유저스페이스 연동 (WDT 및 부팅 성공)

시스템 행(hang)을 감지하고, 부팅 성공을 U-Boot에 다시 알려주는 리눅스 측 설정을 구현합니다.

#### 1. 하드웨어 워치독(WDT) 활성화:

- project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 파일을 엽니다.<sup>10</sup>
- Cadence WDT(&watchdog0)를 활성화합니다.<sup>15</sup>

DTS

```
/include "system-conf.dtsi"
```

```
{
};
```

```
/* Enable Cadence WDT (FPD WDT) for system hang detection */
```

```
&watchdog0 {
    status = "okay";
    reset-on-timeout; /* PMUFW에 시스템 리셋 요청 [15] */
    timeout-sec = ; /* 120초 타임아웃 */
};
```

- PMU 펌웨어 설정 (필수):

- project-spec/meta-user/recipes-bsp/pmu-firmware/ 디렉토리를 생성합니다.
- pmu-firmware\_%.bbappend 파일을 생성합니다.
- vi  
project-spec/meta-user/recipes-bsp/pmu-firmware/pmu-firmware\_%.bbappend
- WDT 리셋을 위한 오류 관리(EM) 모듈을 활성화합니다.<sup>15</sup>

코드 스니펫

```
# Enable Error Management (EM) for WDT Reset
YAML_COMPILER_FLAGS_append = " -DENABLE_EM"
```

## 2. systemd 위치독 연동:

- PetaLinux 2022.1 이상은 systemd를 사용합니다. systemd가 WDT를 자동으로 'pet'(먹이 주기)하도록 설정합니다.<sup>19</sup>
- project-spec/meta-user/recipes-core/systemd/systemd\_%.bbappend 파일을 생성합니다.
- project-spec/meta-user/recipes-core/systemd/systemd/ 디렉토리를 생성합니다.
- system.conf 파일을  
project-spec/meta-user/recipes-core/systemd/systemd/system.conf 경로에 생성합니다.

코드 스니펫

```
# /etc/systemd/system.conf override
[Manager]
RuntimeWatchdogSec=60s
ShutdownWatchdogSec=10min
```

- systemd\_%.bbappend 파일에 다음 내용을 추가합니다:

코드 스니펫

```
FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://system.conf"
```

```
do_install_append() {
```

```
    install -m 0644 ${WORKDIR}/system.conf ${D}${sysconfdir}/systemd/system.conf
}
```

- 작동 원리: systemd는 60초마다 /dev/watchdog0에 신호를 보냅니다.<sup>21</sup> 리눅스 커널이나 유저스페이스가 멈추면 systemd도 멈춥니다. 120초(DTB의 timeout-sec)가 지나면 WDT가 리셋을 트리거합니다.

## 3. fw\_setenv 환경 설정:

- 리눅스에서 U-Boot 환경 변수를 수정하려면 fw\_setenv가 Kria의 QSPI bootenv 파티션을 찾아야 합니다.<sup>22</sup>
- Kria BSP의 PetaLinux 설정(petalinux-config -> Subsystem AUTO Hardware Settings -> Flash Settings)을 확인하여 bootenv 파티션(보통 mtd1 또는 mtd2)의 오프셋과 크기를 확인합니다.
- project-spec/meta-user/recipes-core/base-files/base-files\_%.bbappend 파일을 생성합니다.



- project-spec/meta-user/recipes-core/base-files/base-files/ 디렉토리를 생성합니다.
- fw\_env.config 파일을  
project-spec/meta-user/recipes-core/base-files/base-files/fw\_env.config 경로에 생성합니다. (주의: **MTD** 번호와 오프셋은 **cat /proc/mtd**로 확인 후 조정해야 할 수 있습니다)

코드 스니펫

```
# /etc/fw_env.config for Kria KR260 (QSPI Environment)
# MTD 장치명, 장치 오프셋, 환경 변수 크기 [22, 24]
```

```
# KR260 BSP 기본값 (예시: mtd1)
/dev/mtd1 0x00000000 0x00020000
```

- base-files\_%.bbappend 파일에 다음 내용을 추가합니다:

코드 스니펫

```
FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://fw_env.config"
```

```
do_install_append() {
    install -d ${D}${sysconfdir}
    install -m 0644 ${WORKDIR}/fw_env.config ${D}${sysconfdir}/fw_env.config
}
```

#### 4. "부팅 성공" 확정 (**boot-commit**) 서비스:

- 부팅이 성공적으로 완료되면 U-Boot의 bootcount와 boot\_x\_left 변수를 리셋하는 서비스입니다.<sup>22</sup>
- project-spec/meta-user/recipes-core/ 하위에 boot-commit 디렉토리를 생성합니다.
- boot-commit/boot-commit.bb 레시피 파일을 생성합니다.

코드 스니펫

```
SUMMARY = "A/B Boot Commit Service"
LICENSE = "MIT"
LIC_FILES_CHKSUM =
"file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
```

```
SRC_URI = " \
    file://boot-commit.sh \
    file://boot-commit.service \
"
```

```
inherit systemd
```

```
SYSTEMD_SERVICE_${PN} = "boot-commit.service"
```

```
do_install() {
    install -d ${D}${bindir}
    install -m 0755 ${WORKDIR}/boot-commit.sh ${D}${bindir}/
```

```
install -d ${D}${systemd_system_unitdir}
install -m 0644 ${WORKDIR}/boot-commit.service ${D}${systemd_system_unitdir}/
}
```

```
FILES_${PN} = "${bindir}/boot-commit.sh ${systemd_system_unitdir}/boot-commit.service"
```

- boot-commit/files/ 디렉토리를 생성하고, 그 안에 다음 두 파일을 생성합니다.
- boot-commit/files/boot-commit.service:

```
코드 스니펫
```

```
[Unit]
```

```
Description=Mark Boot as Successful (A/B Boot Commit)
```

```
After=multi-user.target
```

```
Type=oneshot
```

```
ExecStart=/usr/bin/boot-commit.sh
```

```
[Install]
```

```
WantedBy=multi-user.target
```

- boot-commit/files/boot-commit.sh:

```
Bash
```

```
#!/bin/sh
```

```
echo "## Committing A/B Boot..."
```

```
# 1. 커널 cmdline에서 현재 부팅된 슬롯 ID 확인 [9]
```

```
SLOT_ID=$(cat /proc/cmdline | grep -o 'rauc.slot=[^ ]*' | cut -d'=' -f2)
```

```
# 2. 해당 슬롯의 시도 횟수를 3으로 리셋
```

```
if; then
```

```
fw_setenv boot_a_left 3
```

```
elif; then
```

```
fw_setenv boot_b_left 3
```

```
else
```

```
echo "ERROR: Could not determine booted slot. No commit."
```

```
exit 1
```

```
fi
```

```
# 3. WDT 실패 카운터(bootcount)를 0으로 리셋 [3]
```

```
fw_setenv bootcount 0
```

```
echo "## Slot ${SLOT_ID} committed successfully."
```

- 마지막으로, PetaLinux가 이 레시피를 빌드하도록 project-spec/meta-user/conf/user-rootfsconfig 파일에 다음 라인을 추가합니다:

CONFIG\_boot-commit

- petalinux-config -c rootfs를 실행하고, user packages에서 boot-commit이 선택되었는지 확인 후 저장/종료합니다.

## 6단계: 빌드, 패키징 및 SD 카드 프로그래밍

1. 전체 PetaLinux 프로젝트를 빌드합니다.

```
Bash
petalinux-build
```

2. 빌드가 완료되면 images/linux/ 디렉토리에 필요한 파일이 생성됩니다:

- boot.scr (A/B 로직 스크립트)
- Image (커널)
- system.dtb (디바이스 트리)
- rootfs.tar.gz (루트 파일 시스템)

3. SD 카드에 파일 복사 (2단계에서 포맷한 SD 카드 마운트):

- sudo mount /dev/sdc1 /mnt/boot
- sudo mount /dev/sdc2 /mnt/rootfs\_a
- sudo mount /dev/sdc3 /mnt/rootfs\_b

4. 슬롯 A (Golden) 구성:

- boot 파티션에 A 슬롯용 파일 복사:  
Bash  
cd images/linux/  
sudo cp Image /mnt/boot/Image-A  
sudo cp system.dtb /mnt/boot/system-A.dtb  
sudo cp boot.scr /mnt/boot/

- rootfs\_a 파티션에 루트 파일 시스템 압축 해제:

```
Bash
sudo tar -xvzf rootfs.tar.gz -C /mnt/rootfs_a
```

5. 슬롯 B (Update) 구성:

- boot 파티션에 B 슬롯용 파일 복사 (PoC에서는 동일한 이미지 사용):

```
Bash
sudo cp Image /mnt/boot/Image-B
sudo cp system.dtb /mnt/boot/system-B.dtb
```

- rootfs\_b 파티션에 루트 파일 시스템 압축 해제:

```
Bash
sudo tar -xvzf rootfs.tar.gz -C /mnt/rootfs_b
```

6. SD 카드를 마운트 해제합니다.

```
Bash
sync
sudo umount /mnt/boot /mnt/rootfs_a /mnt/rootfs_b
```

## 7단계: PoC 검증 및 장애 시나리오 테스트

SD 카드를 Kria KR260에 삽입하고 전원을 켭니다. UART 터미널(115200)을 연결하여 부팅 로그를 모니터링합니다.

1. 초기 부팅 (슬롯 A):

- U-Boot 로그에 "Attempting to boot slot A..."가 표시됩니다.
- Linux 부팅이 완료되면 boot-commit.service가 실행되고, "Slot A committed successfully" 메시지가 로그에 남습니다.
- fw\_printenv를 실행하여 boot\_a\_left=3이고 bootcount=0인지 확인합니다.

2. 테스트 1: Linux 시스템 행 (WDT 페일오버)

- 부팅이 완료된 후, 커널 패닉을 유도하여 systemd가 WDT를 'pet'하지 못하게 합니다.<sup>26</sup>

```
Bash
echo c > /proc/sysrq-trigger
```

- 시스템이 즉시 패닉 상태가 되고 응답을 멈춥니다.
- 120초(DTB timeout-sec) 후, 하드웨어 WDT가 시스템을 리셋합니다.
- UART 로그에서 U-Boot가 "WARN: Boot failure detected"를 출력하고 boot\_a\_left를 2로 설정하는 것을 확인합니다.
- 시스템은 다시 슬롯 A로 부팅됩니다 (시도 횟수가 남아있으므로).
- 이 테스트를 3회 반복합니다. 3번째 리셋 후, boot\_a\_left는 0이 됩니다.
- 예상 결과: U-Boot가 "Found valid slot B. Tries left: 3"를 출력하고, B 슬롯의 커널(Image-B)과 RootFS(rootfs\_b)로 부팅합니다.

3. 테스트 2: 커널 이미지 손상 (bootm 페일오버)

- 슬롯 B로 부팅된 상태에서, 슬롯 A의 커널을 고의로 손상시킵니다.

```
Bash
# SD 카드 boot 파티션을 마운트
mount /dev/mmcblk1p1 /boot
rm /boot/Image-A
sync
umount /boot
```

```
# 슬롯 A로 부팅하도록 U-Boot 환경 변수 강제 설정
fw_setenv boot_order "A B"
```

```
fw_setenv boot_a_left 3  
reboot
```

- 예상 결과: U-Boot가 "Attempting to boot slot A.."를 시도하지만, "ERROR: Failed to load Kernel (Image-A)!" 메시지를 출력하고 리셋됩니다.
- bootcount 로직에 의해 boot\_a\_left가 차감되며 3회 반복됩니다.
- 3회 실패 후, U-Boot가 자동으로 boot\_order를 B로 전환하고 정상적인 슬롯 B로 부팅하여 시스템을 복구합니다.